[2] Refer back to chapter 3 if you haven't installed mkcert yet.

[3] https://wiki.mozilla.org/Security/Server_Side_TLS

[4] Thomas Pornin, the author of the BearSSL library, has detailed notes on the cost of different TLS cryptographic algorithms at https://bearssl.org/support.html.

[5] ChaCha20-Poly1305 also suffers from nonce reuse problems like GCM, but to a lesser extent. GCM loses all authenticity guarantees after a single nonce reuse, while ChaCha20-Poly1305 only loses these guarantees for messages encrypted with the duplicate nonce.

[6] Support for X25519 has also been added to TLS 1.2 and earlier in a subsequent update, see https://tools.ietf.org/html/rfc8422.

[7] The author of the reference implementation, Jim Schaad, also runs a winery named August Cellars if you are wondering about the domain name.

[8] It's unfortunate that COSE tries to handle both cases in a single class of algorithms. Requiring the expand function for HKDF with a hash function is inefficient when the input is already uniformly random. On the other hand, skipping it for AES is potentially insecure if the input is not uniformly random.

[9] The sharp-eyed among you may notice that this is a variation of the MAC-then-Encrypt scheme that we said in chapter 6 is not guaranteed to be secure. Although this is generally true, SIV mode has a proof of security so is an exception to the rule.

[10] At 4.5MB Bouncy Castle doesn't qualify as a compact implementation, but it shows how AES-SIV can be easily implemented on the server.

[11] The Signal secure messaging service is famous for its "double ratchet" algorithm (https://signal.org/docs/specifications/doubleratchet/), which ensures that a fresh key is derived after every single message.

[12] Real path-traversal exploits are usually more complex than this, relying on subtle bugs in URL parsing routines.